

AgentGarage — Full Roadmap

Date: 2026-09-18

HEAD: e220b37 — CDK skeleton (app + nested stacks)

Modes: mode=demo (public, seed data, TTLs) + mode=private (VPC-internal, admin-only, no seed) from one CDK codebase

Revised architecture: Neptune Serverless (graph) + Bedrock native with BYOK override (world model) + Kinesis (ingest buffer)

Governing rule: a predicted failure is a hypothesis until the real agent reproduces it in the sandbox.

1. Vision (from docs/overview.md)

Autonomous agents pick tools in a loop and branch on what they observe, with broad permissions and weak guardrails. Testing covers a fraction of reachable paths.

AgentGarage maps which behaviours an agent actually exercised, finds reachable but unexplored state-action pairs, predicts outcomes with a world model, replays promising ones against the real agent in isolation, and keeps every confirmed failure as a permanent eval.

Statuses: observed -> predicted -> verified -> protected.

Trust split:

- World model proposes. It cannot write evals.
- Sandbox executes. It cannot reach Bedrock or the world model.
- Bedrock narrates. It cannot change statuses.
- Only the rule check in the sandbox moves predicted to verified.
- Only the eval compiler, given a failed rule log, moves verified to protected.

Demo agent: autonomous refund agent — goal + four tools, no fixed script. Branching is real, so the map is real.

2. Baseline — what is done

Commit	Work
c4a7352	Scaffold repo: schemas/, services/{api,collector,graph,simulate,replay,evals}/, agent/, web/, docs/
918fa9d	One-line README per service
758f7e3	Docs: overview, architecture, demo, ui, cdk
56b30df	Schemas + templates (underscored names)
b47e7cd / 8e1501d	Rename frontier -> gaps -> unexplored
4b2c68f	Mock API handlers returning templates (services/api/app.py), 8 routes
aadcb71	Ignore pycache

Commit	Work
e220b37	CDK skeleton: <code>cdk/app.py</code> + 8 NestedStacks (stubs, bodies land next)

Key specs already locked:

- `schemas/api_routes.json`: GET `/agent/{id}/graph`, GET `/agent/{id}/unexplored`, POST `/agent/{id}/simulate`, POST `/scenarios/{id}/sandbox`, GET `/sandboxes/{id}`, GET `/evals`, POST `/evals/run`, POST `/demo/seed`.
- Templates: `graph_sample.json`, `unexplored_sample.json`, `scenario_19.json` (`sc_19`, `refund_succeeded` + `tool_timeout`, `fault timeout_after_success`, `hypothesis double-refund`), `trace_84f2.json`, `eval_duplicate_refund.json` (status protected).
- Demo flow (`docs/demo.md`): clean runs -> highlight timeout-after-success -> predict double refund -> sandbox verifies `refund_calls=2` -> eval protected -> add idempotency key -> re-run pass.
- UI (`docs/ui.md`): graph-first, dark theme, red = verified, grey = predicted, trace/log/eval one click away, live sandbox logs, eval cards with history, <2s on demo data, Linear polish / Langfuse density.

3. Architecture revisions (user decisions)

3.1 Graph storage: OpenSearch -> Neptune Serverless

- Replace `opensearch.Domain` with Neptune Serverless DB cluster.
- Property graph: vertices state, edges action with `agent_id`, `status`, `count`.
- Query via Gremlin / openCypher Lambda for graph + unexplored endpoints.
- `stacks/search.py` repurposed as `stacks/graph.py` (or rewritten in place + `app.py` import updated).

3.2 Model path: EC2 Qwen-AgentWorld -> Bedrock native + BYOK override

- Delete EC2 `g5.xlarge` + Hugging Face boot pull.
- New `simulation_fn` Lambda: default `bedrock:InvokeModel` in-region; if `agents` row has `model_override {endpoint, secret_arn}`, call customer endpoint via Secrets Manager.
- `agents` table gains `model_endpoint`, `model_secret_arn`, `image_uri`.
- Sandbox gets zero Bedrock/BYOK IAM (trust split preserved).

3.3 Ingest buffer: direct collector -> Kinesis

- Path: ECS OTLP collector -> Kinesis Data Stream (on-demand, KMS) -> Cedar `redact_fn` (EventSourceMapping, batch 100, DLQ) -> S3 traces + DynamoDB sessions.
- Retention: 24h demo / 7d private. Dual-stream optional later.

4. Implementation phases

Phase 0 — Restore + hygiene (1 task)

- Working tree restored from HEAD. Verify `git status` clean.
- Decide: keep Python CDK (`app.py` + `stacks/*.py`); update `docs/cdk.md` repo-map which still references TypeScript `bin/agentgarage.ts`.

Phase 1 — Network + Storage

- `stacks/network.py`: VPC (public + private subnets, 1 NAT), S3/DDB Gateway endpoints, Kinesis Interface endpoint, SGs for Neptune / Fargate / Lambdas. Expose `self.vpc`.
- `stacks/storage.py`: KMS key; 3 S3 buckets (traces, sandbox-logs, evals; versioned, block-public; demo 1-day expiry, private no expiry + deletion protection); 4 DDB on-demand tables (sessions, jobs, evals, agents with BYOK fields; TTL only in demo for sessions/jobs). Secrets Manager: collector key, demo creds.
- Verify: `cdk synth -c mode=demo`, `cdk synth -c mode=private`.

Phase 2 — Graph (Neptune Serverless)

- Cluster Serverless (min ~1 NCU demo, higher min private), IAM-auth, KMS, private subnets, ingress only from query-Lambda / builder SGs.
- Index/seed: state-graph schema; graph-builder Lambda on S3 arrival writes edges.
- Query Lambda serves dashboard. Expose `cluster_endpoint`, `graph_query_fn`.
- Verify: no `AWS::OpenSearch::Domain` in synth; `AWS::Neptune::DBCluster` present.

Phase 3 — Intake (Kinesis buffer)

- `stacks/intake.py`: ECS Fargate collector service + ALB (public + API key in demo; internal ALB in private) -> Kinesis Stream -> `redact_fn` -> S3 + DDB.
- ESM with bisect-on-error + SQS DLQ. CloudWatch lag alarms.
- Expose `collector_url`, `stream`. Outputs: endpoint + secret ARN + stream name.

Phase 4 — Compute + Model path

- `stacks/compute.py`: ECS cluster + Fargate sandbox task defs (no internet egress, killed after run, logs to sandbox-logs bucket). Remove GPU host.
- `simulation_fn`: Bedrock default + BYOK branch, writes predicted scenarios (e.g. `sc_19` shape) to DDB/S3.
- IAM: simulation gets `bedrock:InvokeModel` + scoped `secretsmanager:GetSecretValue`; sandbox gets none for models.
- Expose `cluster`, `sandbox_task`, `simulation_fn`.

Phase 5 — Orchestration

- `stacks/orchestration.py`: EventBridge/S3 rule (trace arrival -> graph builder); Step Functions sandbox sequence: restore start state -> inject fault -> `ecs:RunTask` agent image (default or `agents.image_uri`) -> rule-check Lambda (predicted -> verified) -> eval-compiler Lambda (verified -> protected, writes YAML to evals bucket + DDB index).
- Expose `state_machine`.

Phase 6 — API + Frontend

- `stacks/api.py`: `RestApi` + Cognito authorizer; 8 routes wired to Lambdas (assets from `services/api/`; mocks replaced incrementally). Graph/simulate hit Neptune-query + simulation Lambdas. Least-privilege per function.
- `stacks/frontend.py`: Cognito UserPool (self-signup demo; admin-only + `adminEmail` private) + Amplify app from `web/` with `API_URL = api.url`.

- `app.py`: keep wiring order Network -> Storage -> Graph/Intake/Compute -> Orchestration -> Api -> Frontend.
Add `CfnOutputs`: dashboard URL, collector endpoint + secret ARN, Neptune endpoint, Kinesis stream, Bedrock model IDs. Remove stale OpenSearch / EC2 outputs.

Phase 7 — Demo + hardening

- `POST /demo/seed {runs:5}` -> 5 traces -> dashboard draws observed paths + one unexplored (`refund_succeeded + tool_timeout`) -> simulate predicts double-refund -> `sandbox_verified_fail` (`refund_calls=2`, log in S3) -> eval protected -> fix agent (idempotency key) -> `POST /evals/run` passes.
- UI: paginate everything, stream sandbox logs (no hidden spinners), eval cards link trace/log/file.
- Performance: <2s on demo data.

5. Deploy

Prereqs: AWS admin profile, Node 20 + CDK CLI, Bedrock model access in-region, Neptune + Kinesis quotas (no more `g5.xlarge` quota needed), Amplify/domain (or default URL).

```
cd cdk
pip install -r requirements.txt
cdk bootstrap aws://ACCOUNT/REGION
cdk deploy -c mode=demo
# private:
cdk deploy -c mode=private -c adminEmail=you@company.com
cdk destroy # teardown
```

Bring-your-own-agent (private): emit OTLP with `agent.id`, `agent.version`, `prompt.hash`; register container URI in `agents` table; optionally register BYOK model endpoint + secret.

6. Milestones + acceptance

- M1: synth clean both modes; no OpenSearch/EC2 resources; Neptune + Kinesis + Bedrock IAM present.
- M2: seed 5 traces; graph + unexplored render.
- M3: `sc_19` predicted -> `sb_07 verified_fail` -> eval protected -> pass-after-fix. Demo video needs one failed run + one passing re-run, nothing else.
- M4: private deploy in own account, data stays in-region, admin-only auth.

7. Risks / open items

- Neptune Serverless min-NCU cost vs provisioned — confirm for demo budget.
- Bedrock model allow-list per region; BYOK auth patterns per customer.
- Kinesis retention cost in private (7d) vs MSK later if throughput grows.
- `docs/cdk.md` TypeScript repo-map vs Python reality — update doc in Phase 0.
- `services/{collector,graph,simulate,replay,evals}` still README-only — bodies needed after CDK lands.

8. Repo map (target)

```

cdk/
  app.py          # entry, reads mode context, wires nested stacks
  cdk.json        # {"app": "python app.py", "context": {"mode": "demo"}}
  requirements.txt # aws-cdk-lib, constructs
  stacks/
    network.py    # VPC, subnets, endpoints, SGs
    storage.py    # S3, DDB, KMS, Secrets
    graph.py      # Neptune Serverless + query Lambda (ex-search.py)
    intake.py     # collector ECS + Kinesis + redact Lambda
    compute.py    # Fargate sandbox + Bedrock/BYOK simulation_fn
    orchestration.py # Step Functions + EventBridge
    api.py        # Gateway + Lambdas
    frontend.py   # Amplify + Cognito
docs/roadmap.md  # this file
docs/roadmap.pdf # rendered output
schemas/        # routes + trace/eval/scenario/vocab + templates
services/api/app.py # handlers (mocks -> AWS wiring)

```